

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ОТЧЁТ ПО ЛАБАРАТОРНОЙ РАБОТЕ 6

По дисциплине Web-программирование

Тема работы Работа с сокетами

Обучающийся Надери Мариам

Факультет Факультет инфокоммуникационных технологий

Группа K3321

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в
инфокоммуникационных системах

Обучающийся	<u>16.04.2025</u> (дата)	<u> </u> (подпись)	<u>Надери М.Ш.</u> (Ф.И.О.)
--------------------	-----------------------------	--	--------------------------------

Руководитель	<u> </u> (дата)	<u> </u> (подпись)	<u>Марченко Е.В.</u> (Ф.И.О.)
---------------------	---------------------------------------	--	----------------------------------

Санкт-Петербург
2025 г.

ВВЕДЕНИЕ

Цель: овладеть практическими навыками и умениями реализации web-серверов и использования сокетов.

Оборудование: компьютерный класс.

Программное обеспечение: Python 2.7-3.6, библиотеки Python: sys, socket.

Задания:

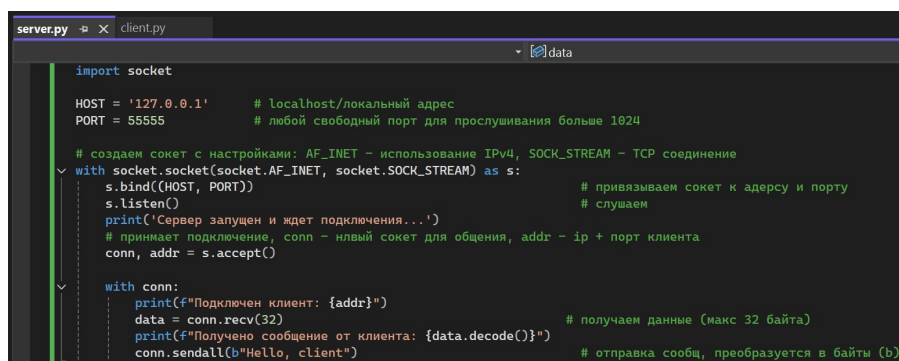
1. Реализовать клиентскую и серверную часть приложения. Клиент отправляет серверу сообщение «Hello, server». Сообщение должно отразиться на стороне сервера. Сервер в ответ отправляет клиенту сообщение «Hello, client». Сообщение должно отобразиться у клиента.
2. Реализовать клиентскую и серверную часть приложения. Клиент запрашивает у сервера выполнение математической операции, параметры, которые вводятся с клавиатуры. Сервер обрабатывает полученные данные и возвращает результат клиенту. Варианты:
 - (a) Теорема Пифагора
 - (b) Решение квадратного уравнения.
 - (c) Поиск площади трапеции.
 - (d) Поиск площади параллелограмма.Вариант выбирается в соответствии с порядковым номером в журнале. Пятый студент получает вариант 1 и т.д.
3. Реализовать серверную часть приложения. Клиент подключается к серверу. В ответ клиент получает http-сообщение, содержащее html-страницу, которую сервер подгружает из файла index.html.
4. Реализовать двухпользовательский или многопользовательский чат. Реализация многопользовательского чата позволяет получить максимальное количество баллов.

ХОД РАБОТЫ

Надо реализовать клиентскую и серверную часть приложения. Клиент отправляет серверу сообщение «Hello, server». Сообщение должно отразиться на стороне сервера. Сервер в ответ отправляет клиенту сообщение «Hello, client». Сообщение должно отобразиться у клиента.

ЗАДАНИЕ 1

Для этого был написан код для сервера (Рис. 1) и для клиента (Рис. 2) на python.



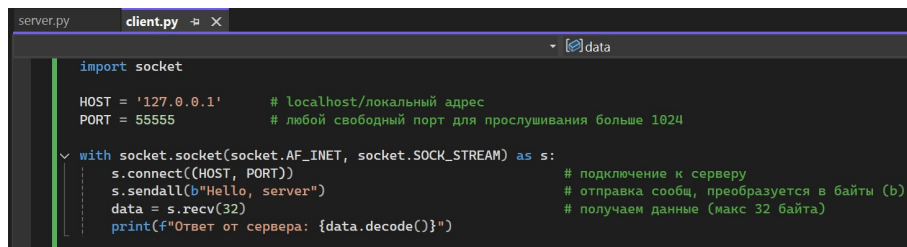
```
server.py  x client.py
import socket

HOST = '127.0.0.1'      # localhost/локальный адрес
PORT = 55555           # любой свободный порт для прослушивания больше 1024

# создаем сокет с настройками: AF_INET - использование IPv4, SOCK_STREAM - TCP соединение
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen()
    print('Сервер запущен и ждет подключения...')
    # принимает подключение, conn - нлвый сокет для общения, addr - ip + порт клиента
    conn, addr = s.accept()

    with conn:
        print(f'Подключен клиент: {addr}')
        data = conn.recv(32)
        print(f'Получено сообщение от клиента: {data.decode()}')
        conn.sendall(b'Hello, client')
```

Рисунок 1



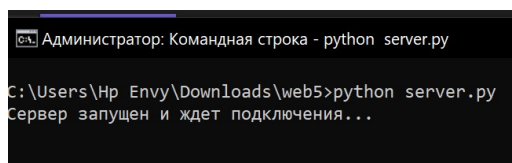
```
server.py  client.py  x
import socket

HOST = '127.0.0.1'      # localhost/локальный адрес
PORT = 55555           # любой свободный порт для прослушивания больше 1024

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    s.sendall(b'Hello, server')
    data = s.recv(32)
    print(f'Ответ от сервера: {data.decode()}')
```

Рисунок 2

На рисунке 3 показано состояние сервера при запуске, но без подключенного клиента. Результат клиента на рисунке 4, а результат сервера на рисунке 5.



```
Администратор: Командная строка - python server.py
C:\Users\Hp Envy\Downloads\web5>python server.py
Сервер запущен и ждет подключения...
```

Рисунок 3

```
Администратор: Командная строка

C:\Users\Hp Envy\Downloads\web5>python client.py
Ответ от сервера: Hello, client

C:\Users\Hp Envy\Downloads\web5>
```

Рисунок 4

```
Администратор: Командная строка

C:\Users\Hp Envy\Downloads\web5>python server.py
Сервер запущен и ждет подключения...
Подключен клиент: ('127.0.0.1', 60188)
Получено сообщение от клиента: Hello, server

C:\Users\Hp Envy\Downloads\web5>
```

Рисунок 5

ЗАДАНИЕ 2

По заданию надо было реализовать клиентскую и серверную часть приложения. Клиент запрашивает у сервера выполнение математической операции, параметры, которые вводятся с клавиатуры. Сервер обрабатывает полученные данные и возвращает результат клиенту. По варианту надо было возвращать решение квадратного уравнения.

Скрипты сервера и клиента на рисунках 6 и 7.

```
server_task2.py  client_task2.py

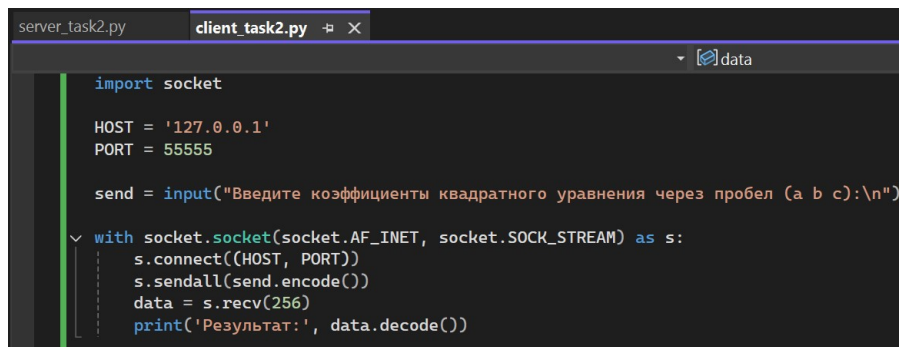
import socket

def solve_quadratic(a, b, c):
    D = b**2 - 4*a*c
    if D < 0:
        return "Нет действительных корней"
    elif D == 0:
        x = -b / (2*a)
        return f"Один корень: {x:.4f}"
    else:
        x1 = (-b - D**0.5) / (2*a)
        x2 = (-b + D**0.5) / (2*a)
        return f"Два корня: {x1:.4f}, {x2:.4f}"

HOST = '127.0.0.1'
PORT = 55555

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen()
    print('Сервер запущен и ждет подключения...')
    conn, addr = s.accept()
    with conn:
        print(f"Подключен клиент: {addr}")
        data = conn.recv(256).decode()
        try:
            a, b, c = map(float, data.split())
            answer = solve_quadratic(a, b, c)
        except:
            answer = "Ошибка ввода данных"
        conn.sendall(answer.encode())
```

Рисунок 6



```
server_task2.py  client_task2.py  X
import socket

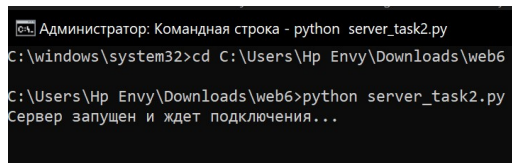
HOST = '127.0.0.1'
PORT = 55555

send = input("Введите коэффициенты квадратного уравнения через пробел (a b c):\n")

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    s.sendall(send.encode())
    data = s.recv(256)
    print('Результат:', data.decode())
```

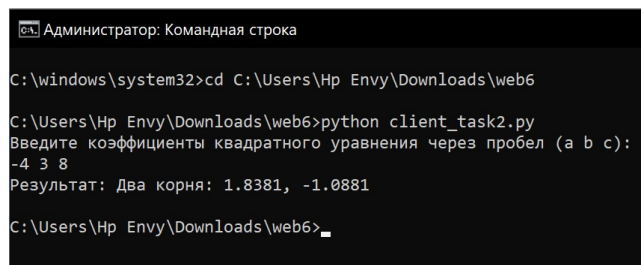
Рисунок 7

На рисунке 8 состояние сервера при запуске. На рисунке 9 клиентский интерфейс, показан результат. На рисунке 10 вид сервера после проведения расчетов.



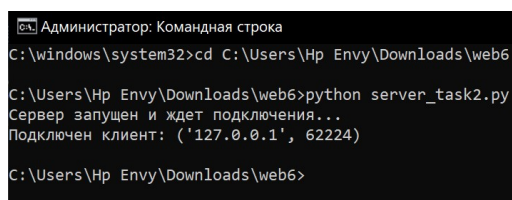
```
Администратор: Командная строка - python server_task2.py
C:\windows\system32>cd C:\Users\Hp Envy\Downloads\web6
C:\Users\Hp Envy\Downloads\web6>python server_task2.py
Сервер запущен и ждет подключения...
```

Рисунок 8



```
Администратор: Командная строка
C:\windows\system32>cd C:\Users\Hp Envy\Downloads\web6
C:\Users\Hp Envy\Downloads\web6>python client_task2.py
Введите коэффициенты квадратного уравнения через пробел (a b c):
-4 3 8
Результат: Два корня: 1.8381, -1.0881
C:\Users\Hp Envy\Downloads\web6>
```

Рисунок 9



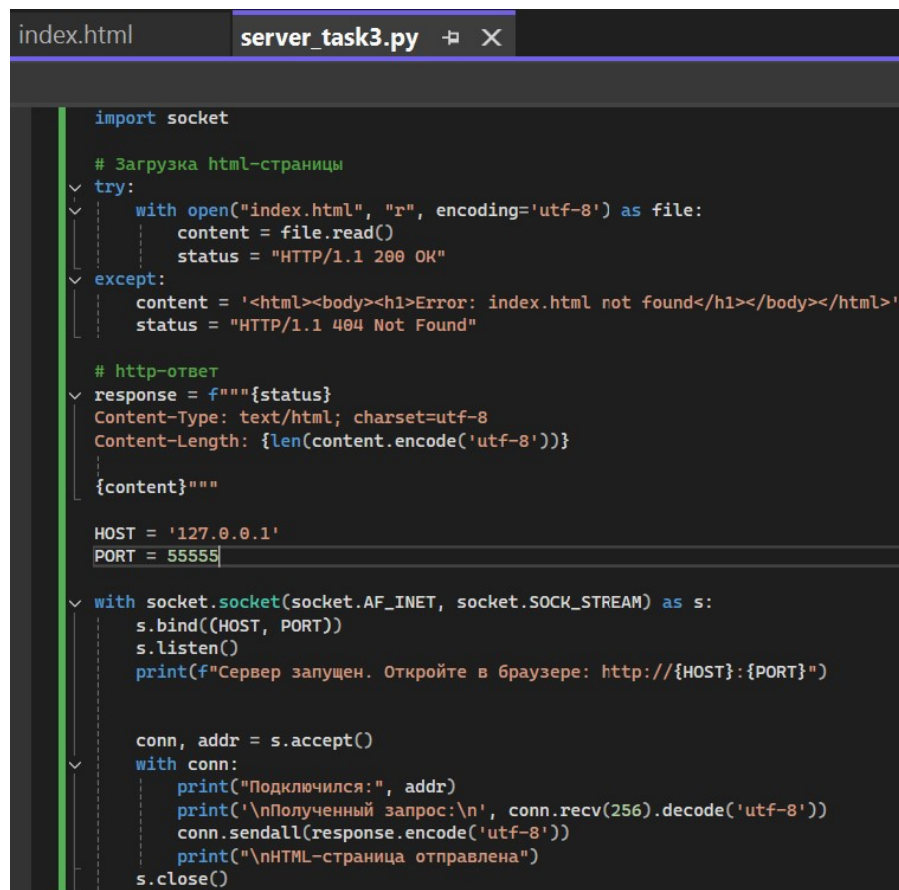
```
Администратор: Командная строка
C:\windows\system32>cd C:\Users\Hp Envy\Downloads\web6
C:\Users\Hp Envy\Downloads\web6>python server_task2.py
Сервер запущен и ждет подключения...
Подключен клиент: ('127.0.0.1', 62224)
C:\Users\Hp Envy\Downloads\web6>
```

Рисунок 10

ЗАДАНИЕ 3

По заданию надо было реализовать серверную часть приложения. Клиент подключается к серверу. В ответ клиент получает http-сообщение, содержащее html-страницу, которую сервер подгружает из файла index.html.

Скрипт работы сервера на рисунке 11. На рисунке 12 скрипт index.html - странички для этого задания.



```
index.html  server_task3.py  X

import socket

# Загрузка html-страницы
try:
    with open("index.html", "r", encoding='utf-8') as file:
        content = file.read()
        status = "HTTP/1.1 200 OK"
except:
    content = '<html><body><h1>Error: index.html not found</h1></body></html>'
    status = "HTTP/1.1 404 Not Found"

# http-ответ
response = f"""{status}
Content-Type: text/html; charset=utf-8
Content-Length: {len(content.encode('utf-8'))}

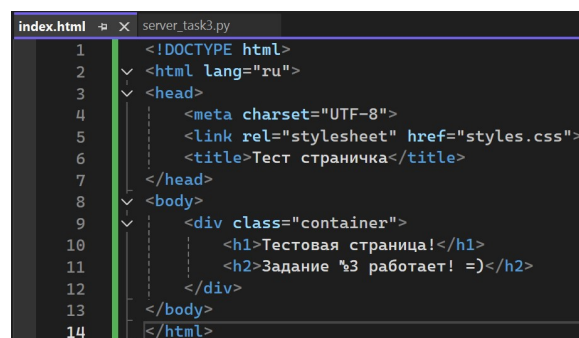
{content}"""

HOST = '127.0.0.1'
PORT = 55555

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen()
    print(f"Сервер запущен. Откройте в браузере: http://{HOST}:{PORT}")

    conn, addr = s.accept()
    with conn:
        print("Подключился:", addr)
        print('\nПолученный запрос:\n', conn.recv(256).decode('utf-8'))
        conn.sendall(response.encode('utf-8'))
        print("\nHTML-страница отправлена")
    s.close()
```

Рисунок 11

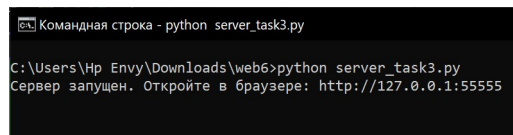


```
index.html  X  server_task3.py

1  <!DOCTYPE html>
2  <html lang="ru">
3  <head>
4      <meta charset="UTF-8">
5      <link rel="stylesheet" href="styles.css">
6      <title>Тест страничка</title>
7  </head>
8  <body>
9      <div class="container">
10         <h1>Тестовая страница!</h1>
11         <h2>Задание '3 работает! =></h2>
12     </div>
13 </body>
14 </html>
```

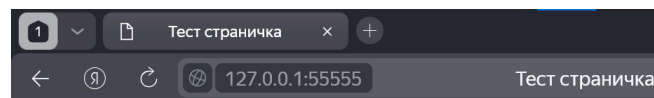
Рисунок 12

На рисунке 13 состояние сервера при запуске. Перейдя по адресу 127.0.0.1:55555 открывается html-страничка (Рис. 14). На рисунке 15 результат на сервере после запуска html-странички.



```
Командная строка - python server_task3.py
C:\Users\Hp Envy\Downloads\web6>python server_task3.py
Сервер запущен. Откройте в браузере: http://127.0.0.1:55555
```

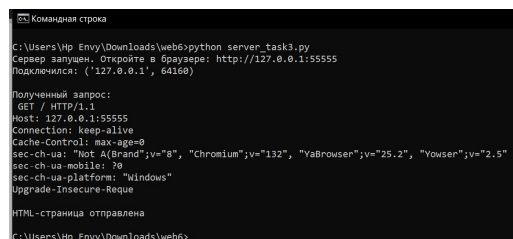
Рисунок 13



Тестовая страница!

Задание №3 работает! =)

Рисунок 14



```
Командная строка
C:\Users\Hp Envy\Downloads\web6>python server_task3.py
Сервер запущен. Откройте в браузере: http://127.0.0.1:55555
Подключился: ("127.0.0.1", 64160)

Полученный запрос:
GET / HTTP/1.1
Host: 127.0.0.1:55555
Connection: keep-alive
Cache-Control: max-age=0
sec-ch-ua: "Not A(Brand";v="8", "Chromium";v="132", "YaBrowser";v="25.2", "Yowser";v="2.5"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Upgrade-Insecure-Require

HTML-страница отправлена
C:\Users\Hp Envy\Downloads\web6>
```

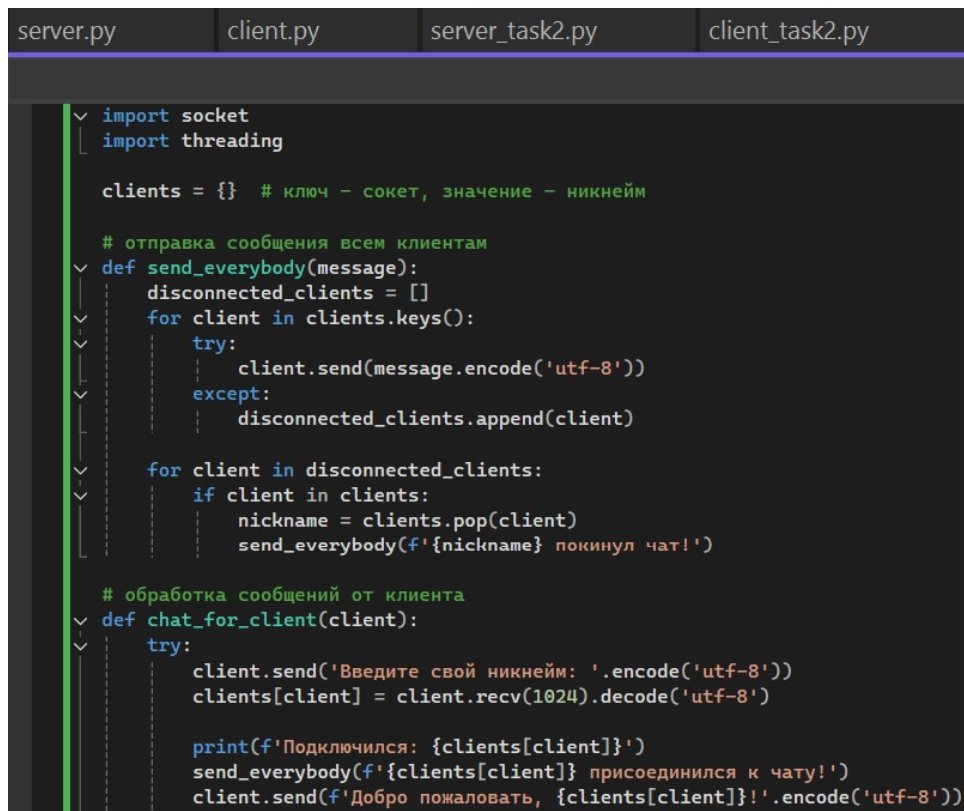
Рисунок 15

ЗАДАНИЕ 4

Надо реализовать двухпользовательский или многопользовательский чат. Реализация многопользовательского чата позволяет получить максимальное количество баллов.

Был реализован многопользовательский чат.

На рисунках 16, 17 - скрипт работы сервера. На рисунках 18, 19 - скрипт работы клиентской части.



```
server.py  client.py  server_task2.py  client_task2.py

import socket
import threading

clients = {} # ключ - сокет, значение - никнейм

# отправка сообщения всем клиентам
def send_everybody(message):
    disconnected_clients = []
    for client in clients.keys():
        try:
            client.send(message.encode('utf-8'))
        except:
            disconnected_clients.append(client)

    for client in disconnected_clients:
        if client in clients:
            nickname = clients.pop(client)
            send_everybody(f'{nickname} покинул чат!')

# обработка сообщений от клиента
def chat_for_client(client):
    try:
        client.send('Введите свой никнейм: '.encode('utf-8'))
        clients[client] = client.recv(1024).decode('utf-8')

        print(f'Подключился: {clients[client]}')
        send_everybody(f'{clients[client]} присоединился к чату!')
        client.send(f'Добро пожаловать, {clients[client]}!'.encode('utf-8'))
```

Рисунок 16


```
client_task4.py  server_task4.py  X

while True:
    message = client.recv(1024).decode('utf-8')
    if not message:
        break
    send_everybody(f'{clients[client]}: {message}')

except (ConnectionResetError, ValueError) as e:
    print(f'{clients[client]} отключился')
finally:
    if client in clients:
        nickname = clients.pop(client)
        send_everybody(f'{nickname} покинул чат!')
    client.close()

HOST = '127.0.0.1'
PORT = 55555

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind((HOST, PORT))
    s.listen()
    print('Сервер запущен и ждет подключения...')

while True:
    conn, addr = s.accept()
    print(f"Подключен клиент: {addr}")

    # Запуск потока для клиента
    thread = threading.Thread(target=chat_for_client, args=(conn,))
    thread.start()
```

Рисунок 17

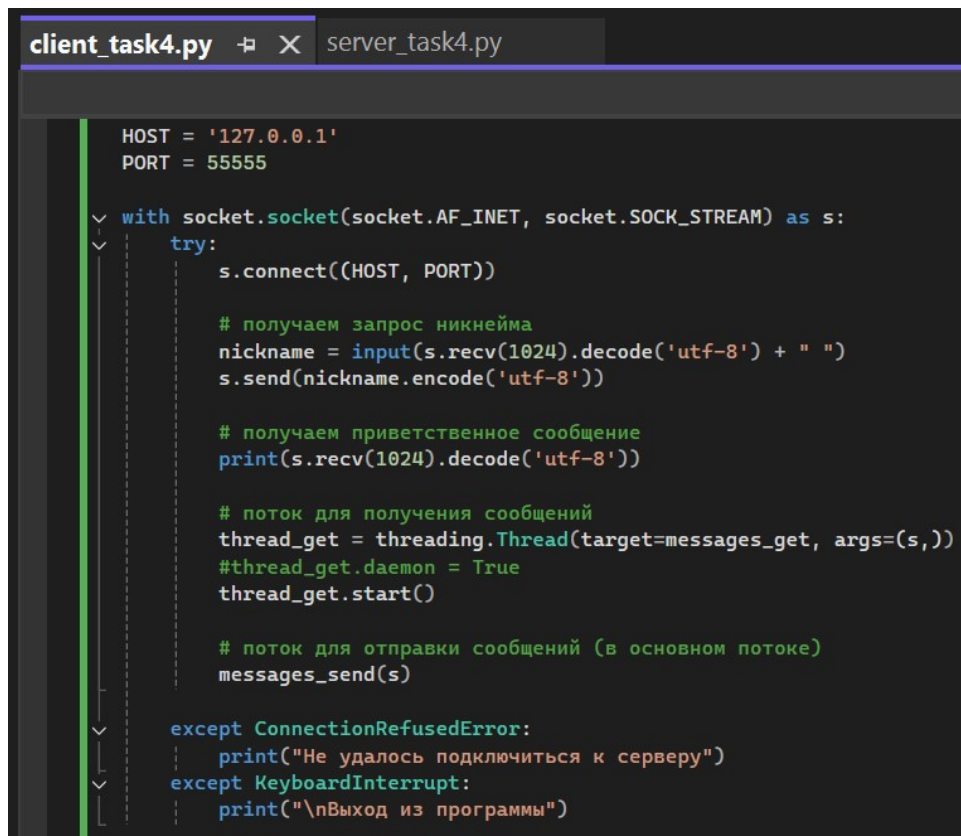
```
client_task4.py  X  server_task4.py

import socket
import threading

# получение сообщений от сервера
def messages_get(client):
    while True:
        try:
            message = client.recv(1024).decode('utf-8')
            if not message:
                print("\nCоединение с сервером разорвано")
                client.close()
                break
            print(message)
        except (ConnectionResetError, OSError):
            print("\nCоединение с сервером разорвано")
            client.close()
            break

# отправка сообщений на сервер
def messages_send(client):
    try:
        while True:
            message = input()
            if message.lower() == 'exit':
                client.close()
                break
            client.send(message.encode('utf-8'))
    except (ConnectionResetError, OSError):
        pass
```

Рисунок 18



```
client_task4.py  X server_task4.py

HOST = '127.0.0.1'
PORT = 55555

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    try:
        s.connect((HOST, PORT))

        # получаем запрос никнейма
        nickname = input(s.recv(1024).decode('utf-8') + " ")
        s.send(nickname.encode('utf-8'))

        # получаем приветственное сообщение
        print(s.recv(1024).decode('utf-8'))

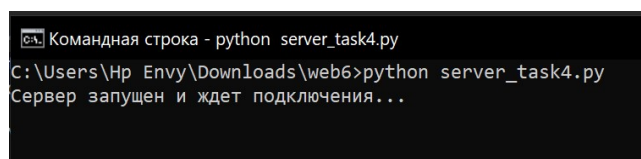
        # поток для получения сообщений
        thread_get = threading.Thread(target=messages_get, args=(s,))
        #thread_get.daemon = True
        thread_get.start()

        # поток для отправки сообщений (в основном потоке)
        messages_send(s)

    except ConnectionRefusedError:
        print("Не удалось подключиться к серверу")
    except KeyboardInterrupt:
        print("\nВыход из программы")
```

Рисунок 19

На рисунке 20 показано состояние сервера после запуска. На рисунке 21 показано как ведет себя чат при подключении трех пользователей. Рисунок 22 показывает случай при выходе одного из участников, а на рисунке 23 видно, что все три пользователя успешно покинули чат. Сообщения также работают.



```
Командная строка - python server_task4.py
C:\Users\Hp Envy\Downloads\web6>python server_task4.py
Сервер запущен и ждет подключения...
```

Рисунок 20

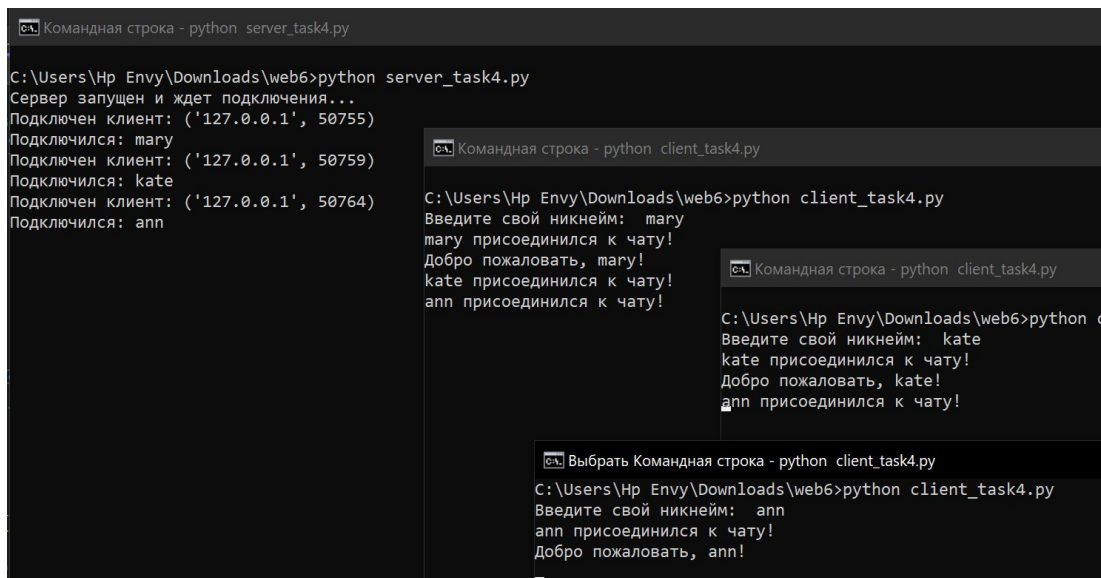


Рисунок 21

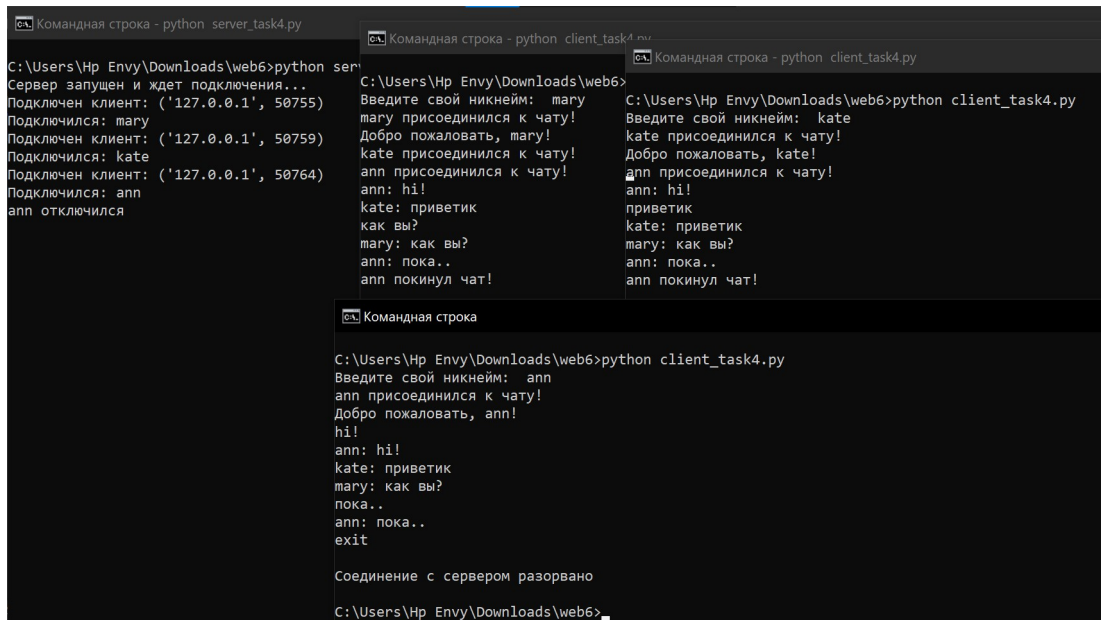


Рисунок 22

```
Командная строка - python server_task4.py

C:\Users\Hp Envy\Downloads\web6>python server_task4.py
Сервер запущен и ждет подключения...
Подключен клиент: ('127.0.0.1', 50755)
Подключился: mary
Подключен клиент: ('127.0.0.1', 50759)
Подключился: kate
Подключен клиент: ('127.0.0.1', 50764)
Подключился: ann
ann отключился
kate отключился
mary отключился

Командная строка

C:\Users\Hp Envy\Downloads\web6>python client_task4.py
Введите свой никнейм: mary
mary присоединился к чату!
Добро пожаловать, mary!
kate присоединился к чату!
ann присоединился к чату!
ann: hi!
kate: приветик
как вы?
mary: как вы?
ann: пока..
ann покинул чат!
kate: ладно.. я тоже пойду
kate покинул чат!
exit

Соединение с сервером разорвано

C:\Users\Hp Envy\Downloads\web6>

Командная строка

C:\Users\Hp Envy\Downloads\web6>python client_task4.py
Введите свой никнейм: kate
kate присоединился к чату!
Добро пожаловать, kate!
ann присоединился к чату!
ann: hi!
приветик
kate: приветик
mary: как вы?
ann: пока..
ann покинул чат!
ладно.. я тоже пойду
kate: ладно.. я тоже пойду
exit

Соединение с сервером разорвано

C:\Users\Hp Envy\Downloads\web6>
```

Рисунок 23